Zaawansowane Programowanie Internetowe

(studia niestacjonarne)

REST, REST API, ASP.NET Core Web API

Zawartość instrukcji:

1. Wstęp do REST oraz REST API.
2. Web API.
3. ASP.NET Core Web API
4. Zadanie

dr inż. Rafał Grycuk

1. Wstęp do REST oraz REST API

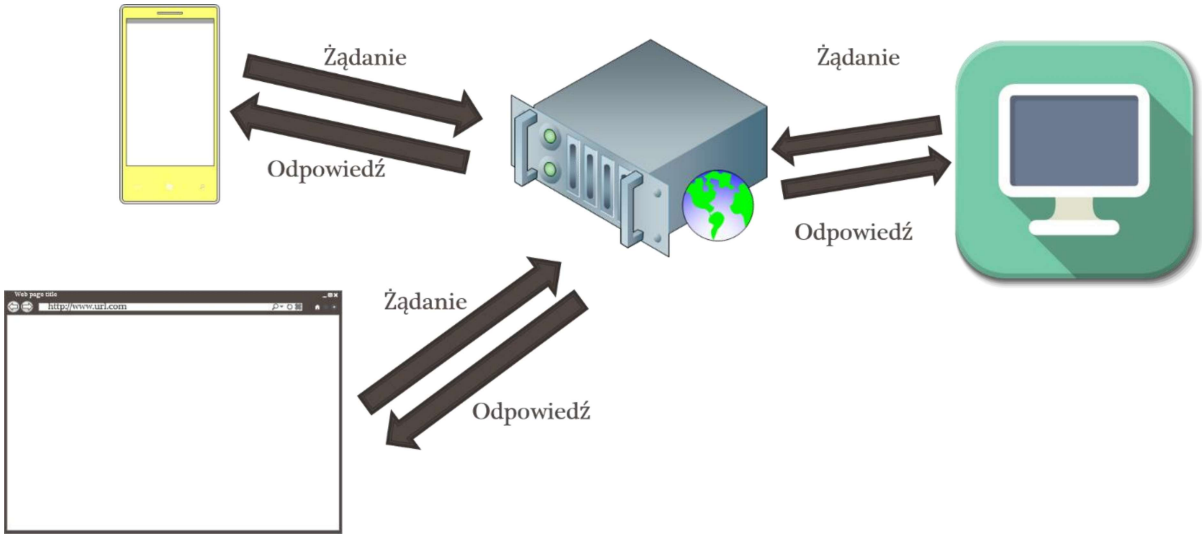
REST (ang. **Re**presentational **S**tate **T**ransfer) – to styl architektoniczny aplikacji zapewniajacy reguły wymiany informacji między systemami komputerowymi. Wspomniane reguły opisują również metody dostępu do udostepnianych zasobów. Nieodłącznym terminem stosowanym wraz z REST jest **API** (ang. **A**pplication **P**rogramming **I**nterface). Jest to zbiór zasad opisujących komunikację pomiędzy jej uczestnikami – zwykle klientem i serwerem, lecz obecnie klientem może być również inny serwer. Inaczej mówiąc API określa metody oraz zasady komunikacji, defininiuje w jaki sposób należy uzyskać dostęp do zasobów oraz w jaki sposób klient uzyska dane. API zwykle jest powiązane z konkretnym protokołem (np. HTTP/S) Natomiast REST jest to styl architektoniczy określający kształt API. REST nie jest bezpośrednio związany z konkretnym protokołem, zbiorem narzedzi czy biblioteką. Twórca REST Roy Fielding zdefiniował następujące elementy architektury REST:

- architektura klient-serwer,
- serwer bezstanowy,
- buforowanie,
- ujednolicony interfejs,
- system warstwowy.

REST API (RESTful API) – jest to wspomniany interfejs programistyczny (API) oparty o REST określający konkretne narzędza, usługi lub oprogramowanie oparte na stylu architektonicznym REST. Chociaż REST może być używany w prawie każdym protokole, zwykle korzysta z protokołu HTTP/S, w połączeniu interfejsem użytkownika (API). Główną zaletą interfejsów RESTful API jest to, że oferują większą elastyczność. W interfejsach tego typu dane nie są ograniczone do zasobów ani metod. Dlatego może wykonywać wiele typów wywołań, zwracać różne formaty danych, a nawet zmieniać strukturę.

2. Web API

Web API w zasadzie to framework, który jest wykorzystywany do tworzenia interfejsów API przy pomocy HTTP/S. Dzięki temu możemy zbudować web service do którego można uzyskać dostęp za pomocą protokołu HTTP/S. Należy zauważyć, że Web API to pewna ogólna koncepcja, a nie konkretna technologia. Developerzy mogą tworzyć interfejsy API przy użyciu szerokiego wachlarza technologii, takich jak między innymi .NET, Java, Node.js, PHP, etc. Interfejs Web API może być RESTful lub nie. Zawiera on specyfikację protokołu HTTP, a zatem obejmuje takie pojęcia, jak buforowanie, identyfikatory URI, wersjonowanie, nagłówki żądań i odpowiedzi i różne formaty danych. Pojęcie Web API jest bardzo często mylone z Web Services, lecz nie jest to precyzyjny termin, gdyż w ten sam sposób oficjalnie nazywają się usługi oparte o SOAP API. Bezpieczniej i poprawniej jest stosować termin HTTP API zamiast Web Service.



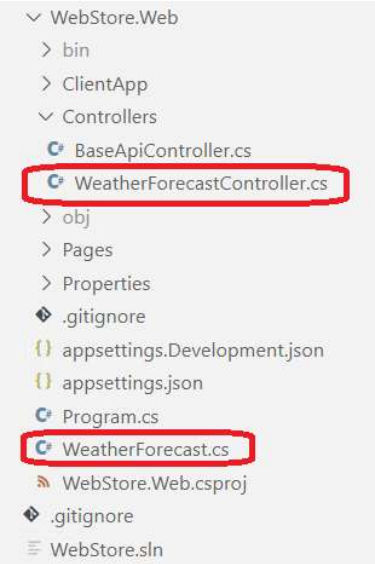
Metoda	Opis
GET	Pobranie zasobu
POST	Dodawanie nowego zasobu
PUT	Modyfikowanie istniejącego zasobu
PATCH	Częściowa modyfikacja istniejącego zasobu
DELETE	Usunięcie zasobu
HEAD	Pobranie metadanych
OPTIONS	Informacje o opcjach i wymaganiach istniejących w kanale komunikacyjnym.
TRACE	Diagnostyka, analiza kanału komunikacyjnego.
CONNECT	Żądanie przeznaczone dla serwerów pośredniczących pełniących funkcje tunelowania.

Kod	Opis	Kod	Opis
200	OK	400	Bad request
201	Created	401	Not authorized
204	Accepted	403	Forbidden
302	Found	404	Not found
304	Not Modified	405	Method not allowed
307	Temp redirect	409	Conflict
308	Perm redirect	412	Precondition Failed
		500	Internal Error

3. ASP.NET Core Web API

Jako że React może działa głównie po stronie klienta, twórcy Reacta zalecają wykorzystywanie szablonu create-react-app. To rozwiązanie stosuje się do dziś, dzieląc całą aplikację na część backendową (REST API) oraz frontendową (React). W takim przypadku, aby aplikacja działała należało najpierw uruchomić aplikację backendową (ASP.NET Core Web API) a następnie poprzez npm oraz webpack aplikację Reactową (frontend). Ponadto w celu poprawnej komunikacji pomiędzy obiema aplikacjami należy również włączyć mechanizm CORS. W celu rozwiązania powyższych niedogodności Microsoft wbudował szablon **create-react-app** w dedykowany Reactowi szablon .NET. Na pierwszych zajęciach przy tworzeniu projektu Web wybierany był właśnie ten szablon. Teraz nadszedł czas, aby rozpocząć tworzenie elementów frontendu a następnie zasilić te elementy danymi pobranymi z Web API. W celu przygotowania projektu **WebStore.Web** należy postępować wg. poniższych kroków:

1) Proszę usunąc poniższe pliki:



2) W kolejnym kroku proszę zainstalować poniższe biblioteki dla projektu **WebStore.Web**:

```
dotnet add WebStore.Web package Microsoft.AspNetCore.Authentication.JwtBearer --version 6.0.9

dotnet add WebStore.Web package Newtonsoft.Json

dotnet add WebStore.Web package Microsoft.AspNetCore.Mvc.NewtonsoftJson --version 6.0.9
```

3) Następnie proszę wkleić poniższą zawartość do pliku **appsettings.json** w projekcie **WebStore.Web** a następnie podmienić {imie_nawisko} na swoje imię i nazwisko, np. Jan_Kowalski

```
{
  "ConnectionStrings": {
    "DefaultConnection":
"Server=(localdb)\\mssqllocaldb;Database={imie_nazwisko}AppDb;Trusted_Connection=True;MultipleActiveResultSets=tru
e"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft": "Warning",
      "Microsoft.Hosting.Lifetime": "Information"
    }
  },
  "JwtOptions": {
    "Issuer": "SuperAwesomeTokenServer",
    "Audience": "http://www.superawesomeprovider.com/",
    "SecretKey": "M6Lc*rmr5s7?_YE__pddasll41&*#)&nq9$vR87T*VA6Dt6*z&Cz4",
    "TokenExpirationMinutes": 20
  },
  "AllowedHosts": "*"
}
```

4) Kolejno proszę podmienić zawartość pliku **Program.cs** w projekcie **WebStore.Web**

```
using System.Text;
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.AspNetCore.Identity;
using Microsoft.EntityFrameworkCore;
using Microsoft.IdentityModel.Tokens;
using Microsoft.OpenApi.Models;
using WebStore.DAL.EF;
using WebStore.Model.DataModels;
using WebStore.Services.ConcreteServices;
using WebStore.Services.Configuration.Profiles;
using WebStore.Services.Interfaces;
using WebStore.ViewModels.VM;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddAutoMapper(typeof(MainProfile));
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")) //here you can define a database type.
);
builder.Services.Configure<JwtOptionsVm>(options => builder.Configuration.GetSection("JwtOptions").Bind(options));
builder.Services.AddIdentity<User, IdentityRole<int>>(o =>
{
    o.Password.RequireDigit = false;
    o.Password.RequireUppercase = false;
    o.Password.RequireLowercase = false;
    o.Password.RequireNonAlphanumeric = false;
    o.User.RequireUniqueEmail = false;
}).AddRoleManager<RoleManager<IdentityRole<int>>>()
.AddUserManager<UserManager<User>>()
.AddEntityFrameworkStores<ApplicationDbContext>()
.AddDefaultTokenProviders();
builder.Services.AddAntiforgery(options => options.HeaderName = "X-XSRF-TOKEN");
builder.Services.AddTransient(typeof(ILogger), typeof(Logger<Program>));

builder.Services.AddAuthentication(options =>
{
    options.DefaultAuthenticateScheme = JwtBearerDefaults.AuthenticationScheme;
    options.DefaultChallengeScheme = JwtBearerDefaults.AuthenticationScheme;
})
.AddJwtBearer(JwtBearerDefaults.AuthenticationScheme, options =>
{
    options.TokenValidationParameters = new TokenValidationParameters
    {
        ValidateAudience = false,
        ValidateIssuer = false,
```

```
        ValidateIssuerSigningKey = true,
        IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(builder.Configuration["JwtOptions:SecretKey"])),
        ValidateLifetime = true,
        ClockSkew = TimeSpan.FromMinutes(5)
    };
});
builder.Services.AddControllersWithViews().AddNewtonsoftJson(options =>
options.SerializerSettings.ReferenceLoopHandling = Newtonsoft.Json.ReferenceLoopHandling.Ignore
);
builder.Services.AddSwaggerGen(opt =>
{
    opt.SwaggerDoc("v1", new OpenApiInfo { Title = "WebStore API", Version = "v1" });
    opt.AddSecurityDefinition("Bearer", new OpenApiSecurityScheme
    {
        In = ParameterLocation.Header,
        Description = "Please enter token",
        Name = "Authorization",
        Type = SecuritySchemeType.Http,
        BearerFormat = "JWT",
        Scheme = "bearer"
    });
    opt.AddSecurityRequirement(new OpenApiSecurityRequirement
    {
        {
          new OpenApiSecurityScheme
          {
            Reference = new OpenApiReference
            {
              Type=ReferenceType.SecurityScheme,
              Id="Bearer"
            }
          },
          new string[]{}
        }
    });
});
var app = builder.Build();
app.UseSwagger();
app.UseSwaggerUI(c => c.SwaggerEndpoint("/swagger/v1/swagger.json", "WebStore API v1"));
// Configure the HTTP request pipeline.
if (!app.Environment.IsDevelopment())
{
    app.UseDeveloperExceptionPage();
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseRouting();
app.UseAuthentication();
app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller}/{action=Index}/{id?}");

app.MapFallbackToFile("index.html"); ;

app.Run();
```

5) Następnie w projekcie **WebStore.ViewModels/VM** proszę dodać dwa view modele o nazwach: **JwtIssuerVm.cs** oraz **LoginUserVm.cs:**

```
namespace  WebStore.ViewModels.VM;
public class JwtOptionsVm
{
    public string Issuer { get; set; } = default!;
    public string Audience { get; set; } = default!;
    public string SecretKey { get; set; }= default!;
    public int TokenExpirationMinutes { get; set; }
}
```

```
using System.ComponentModel.DataAnnotations;
namespace  WebStore.ViewModels.VM;
public class LoginUserVm
{
    [Required]
    [StringLength(50, ErrorMessage = "The {0} must be at least {2} and at max {1} characters long.", MinimumLength
= 6)]
    [Display(Name = "Email")]
    public string Login { get; set; } = default!;

    [Required]
```

```
    [StringLength(100, ErrorMessage = "The {0} must be at least {2} and at max {1} characters long.",
MinimumLength = 6)]
    [DataType(DataType.Password)]
    public string Password { get; set; }  = default!;
}
```

6) Kolejny krok dotyczy kontrolerów. Analogicznie jak poprzednio proszę w projekcie **WebStore.Web/Controllers** stworzyć kontroler bazowy **BaseApiController.cs** a następnie wkleić poniższy kod:

```
using AutoMapper;
using Microsoft.AspNetCore.Mvc;

namespace WebStore.Web.Controllers;

[ApiController]
[Produces("application/json")]
[Route("api/[controller]")]
public abstract class BaseApiController : Controller
{
    protected readonly ILogger Logger;
    protected readonly IMapper Mapper;
    protected BaseApiController(ILogger logger, IMapper mapper)
    {
        Logger = logger;
        Mapper = mapper;
    }
}
```

7) Nasze **Web API** będzie wymagało authentykacji oraz autoryzacji użytkowników. Stąd należy stworzyć kontroler **WebStore/Controllers/AccountApiController.cs** a następnie wkleić poniższy kod:

```
using System.IdentityModel.Tokens.Jwt;
using System.Security.Claims;
using System.Text;
using AutoMapper;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc;
using Microsoft.Extensions.Options;
using Microsoft.IdentityModel.Tokens;
using WebStore.ViewModels.VM;
using WebStore.Model.DataModels;

namespace WebStore.Web.Controllers;

public class AccountApiController : BaseApiController
{
    private readonly SignInManager<User> _signInManager;
    private readonly UserManager<User> _userManager;
    private readonly JwtOptionsVm _jwtOptions;

    public AccountApiController(ILogger logger,
        IMapper mapper,
        IOptions<JwtOptionsVm> jwtOptions,
        SignInManager<User> signInManager, UserManager<User> userManager) : base(logger, mapper)
    {
        _jwtOptions = jwtOptions.Value;
        _signInManager = signInManager;
        _userManager = userManager;
    }


    [AllowAnonymous]
    [HttpPost("[action]")]
    public async Task<IActionResult> Login([FromForm] LoginUserVm applicationUser)
    {
        try
        {
            if (applicationUser == null ||
                ModelState == null ||
                !ModelState.IsValid)
            {
                Logger.LogInformation($"Model is invalid");
                return BadRequest("Invalid credentials");
            }
            var result = await _signInManager.PasswordSignInAsync(applicationUser.Login, applicationUser.Password, true, false);
            if (result.Succeeded == false)
            {
                Logger.LogInformation($"Invalid username ({applicationUser.Login}) or password ({applicationUser.Password})");
```

```
                return BadRequest("Invalid credentials");
            }

            var user = await _userManager.FindByEmailAsync(applicationUser.Login);
            var userRoles = await _userManager.GetRolesAsync(user);
            var claims = new List<Claim> {
                    new Claim (ClaimTypes.Name, applicationUser.Login),
                    new Claim (JwtRegisteredClaimNames.Nbf,
                    new DateTimeOffset (DateTime.Now).ToUnixTimeSeconds ().ToString ()),
                    new Claim (JwtRegisteredClaimNames.Exp,
                    ((long) ((DateTime.Now.AddMinutes (_jwtOptions.TokenExpirationMinutes) - new DateTime (1970, 1, 1, 0, 0,
0)).TotalSeconds)).ToString ())
                };
            claims.AddRange(userRoles.Select(ur => new Claim(ClaimTypes.Role, ur)));

            var token = new JwtSecurityToken(
                new JwtHeader(new SigningCredentials(
                    new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_jwtOptions.SecretKey)),
                    SecurityAlgorithms.HmacSha256)),
                new JwtPayload(claims));

            var encodedJwt = new JwtSecurityTokenHandler().WriteToken(token);
            var response = new
            {
                access_token = encodedJwt,
                expires_in = token.ValidTo.ToString("yyyy-MM-ddTHH:mm:ss")
            };
            return Ok(response);
        }
        catch (Exception ex)
        {
            Logger.LogError(ex, ex.Message);
            return BadRequest("Error occurred");
        }
    }
}
```

Typowym zdaniem w aplikacjach webowych jest obsługa danego zasobu webowego w ramach operacji C.R.U.D. (Create, Read, Update, Delete). Jeżeli weźmiemy pod uwagę zasób **Product**, należy umożliwić użytkownikom obsługę następujących funkcjonalności:

- dodanie nowego produktu (**C**),
- wyświetlenie danych produktu (**R**),
- modyfikację istniejącego produktu (**U**),
- usunięcie / ukrycie wybranego produktu (**D)**.

W tym celu proszę wykonać poniższe kroki:

8) Proszę stworzyć nowy kontroler o nazwie **WebStore/Controllers/ProductApiController.cs**

```
using AutoMapper;
using Microsoft.AspNetCore.Mvc;
using WebStore.Services.Interfaces;
using WebStore.ViewModels.VM;

namespace WebStore.Web.Controllers;

public class ProductApiController : BaseApiController
{
    private readonly IProductService _productService;

    public ProductApiController(ILogger logger, IMapper mapper,
                            IProductService productService)
                            : base(logger, mapper)
    {
        _productService = productService;
    }

    [HttpGet]
    public IActionResult Get()
    {
        try
        {
            var products = _productService.GetProducts();
            return Ok(products);
        }
        catch (Exception ex)
        {
```

```
            Logger.LogError(ex, ex.Message);
            return StatusCode(500, "Error occured");
        }
    }

    [HttpGet("{id:int:min(1)}")]
    public IActionResult Get(int id)
    {
        try
        {
            var product = _productService.GetProduct(p => p.Id == id);
            return Ok(product);
        }
        catch (Exception ex)
        {
            Logger.LogError(ex, ex.Message);
            return StatusCode(500, "Error occured");
        }
    }

    [HttpPut]
    public IActionResult Put([FromBody] AddOrUpdateProductVm addOrUpdateProductVm)
    {
        return PostOrPutHelper(addOrUpdateProductVm);
    }

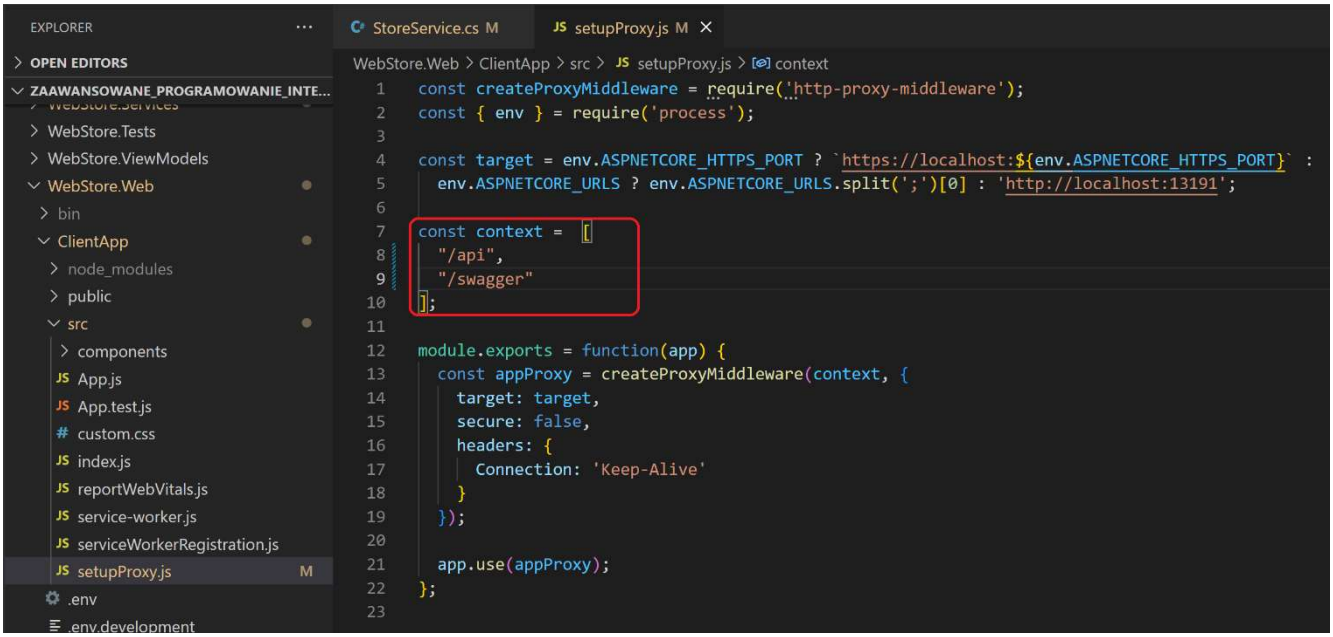
    [HttpPost]
    public IActionResult Post([FromBody] AddOrUpdateProductVm addOrUpdateProductVm)
    {
        return PostOrPutHelper(addOrUpdateProductVm);
    }

    [HttpDelete("{id:int:min(1)}")]
    public IActionResult Delete(int id)
    {
        try
        {
            var result = _productService.DeleteProduct(p => p.Id == id);
            return Ok(result);
        }
        catch (Exception ex)
        {
            Logger.LogError(ex, ex.Message);
            return StatusCode(500, "Error occured");
        }
    }

    private IActionResult PostOrPutHelper(AddOrUpdateProductVm addOrUpdateProductVm)
    {
        try
        {
            if (!ModelState.IsValid)
                return BadRequest(ModelState);
            return Ok(_productService.AddOrUpdateProduct(addOrUpdateProductVm));
        }
        catch (Exception ex)
        {
            Logger.LogError(ex.Message, ex);
            return StatusCode(500, "Error occured");
        }
    }

}
```

9) Ostatnim krokiem przed uruchomieniem projektu jest wyłączenie scieżek **/api** oraz **/swagger** z routingu aplikacji reactowej. Należy to zrobić w pliku **setupProxy.js** dodając te ścieżki.



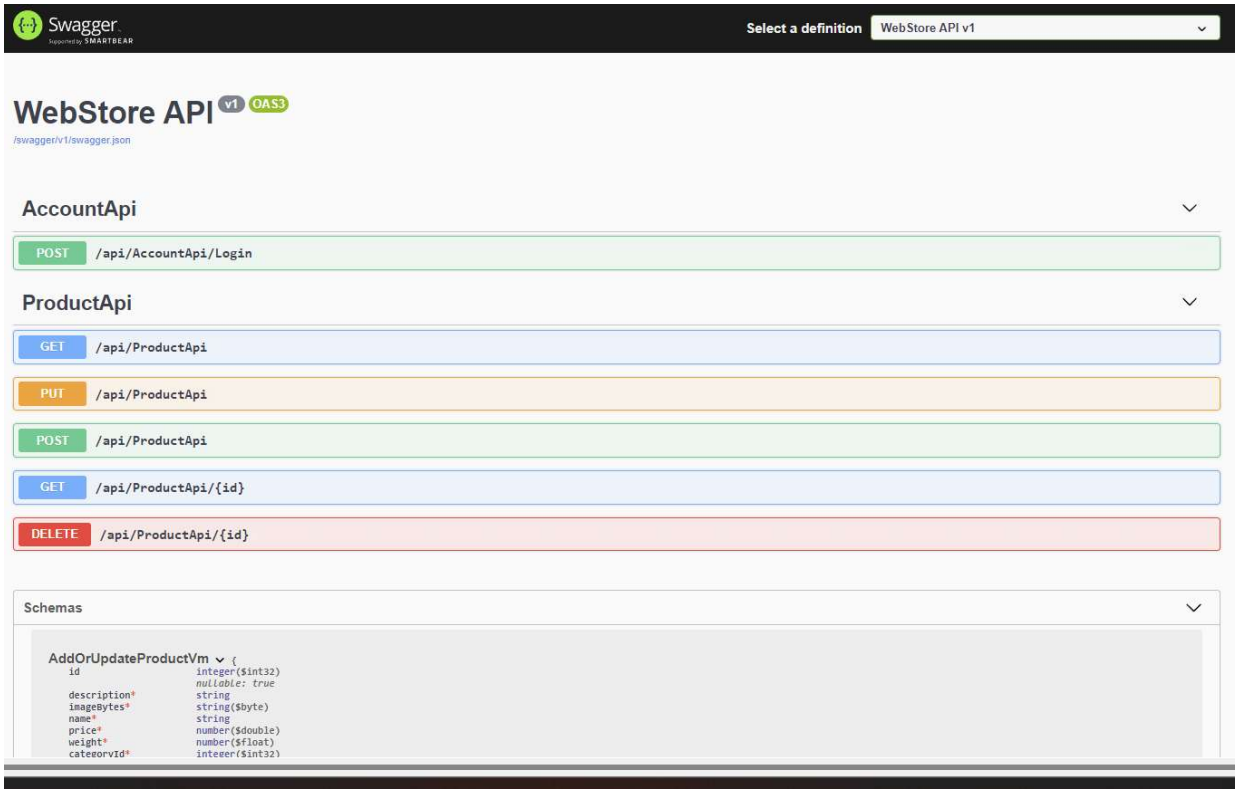
10) W kolejnym kroku proszę uruchomić projekt:

```
dotnet run --project WebStore.Web
```

Posiadając skonfigurowane Web API możemy testować poszczególne enpointy, czyli zasoby URL. W tym celu będziemy potrzebowali klienta HTTP pozwalającego na przygotowanie oraz wysyłanie żądań do serwera z Web API poprzez protokół HTTP. Jednym z domyślnie wbudowanych narzędzi jest Swagger. Jest on automatycznie dodany do projektu stąd nie potrzeba jest go instalować. Aby móc korzystać ze Swaggera, należy przejść pod poniższy address

https://localhost:44433/swagger/index.html

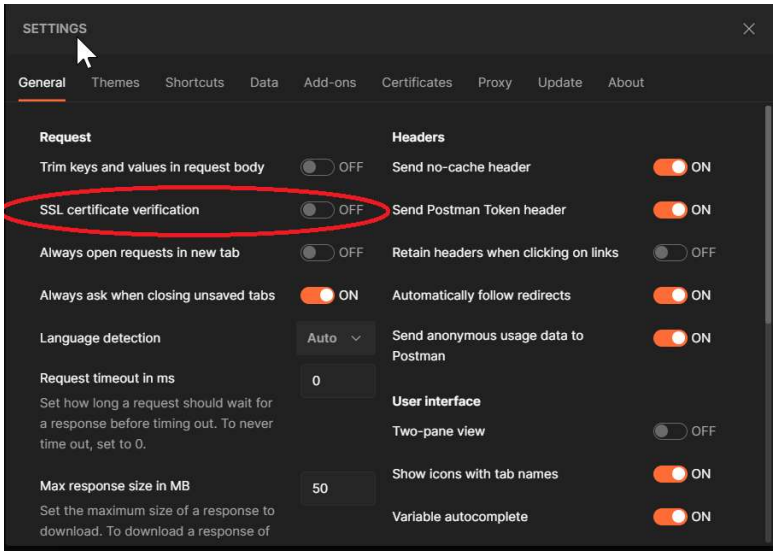
W efekcie powiniśmy uzyskać dostęp nie tylko do klienta HTTP ale również do dynamicznie stworzonej dokumentacji.



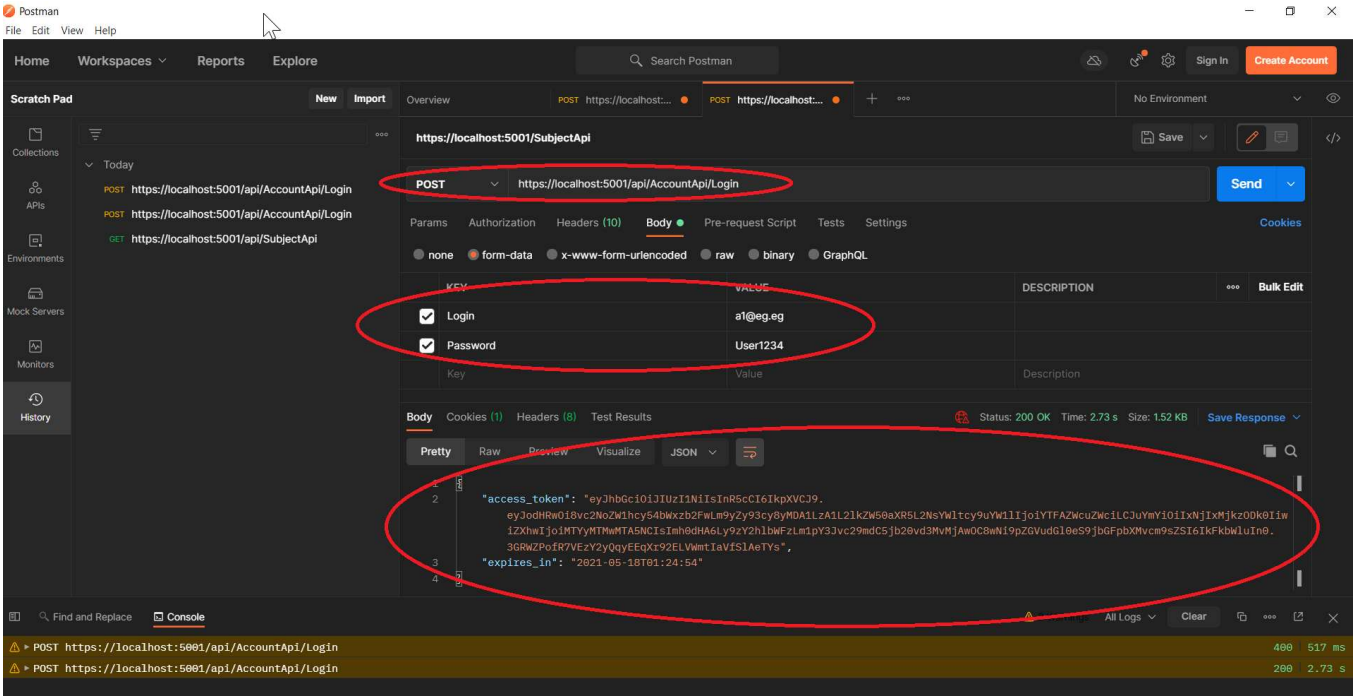
Jednym z popularnieszych klientów HTTP jest Postman. Aby go pobrać należy przejść do poniższego linku, a następnie wybrać odpowiednią dla siebie platformę i zainstalować aplikację.

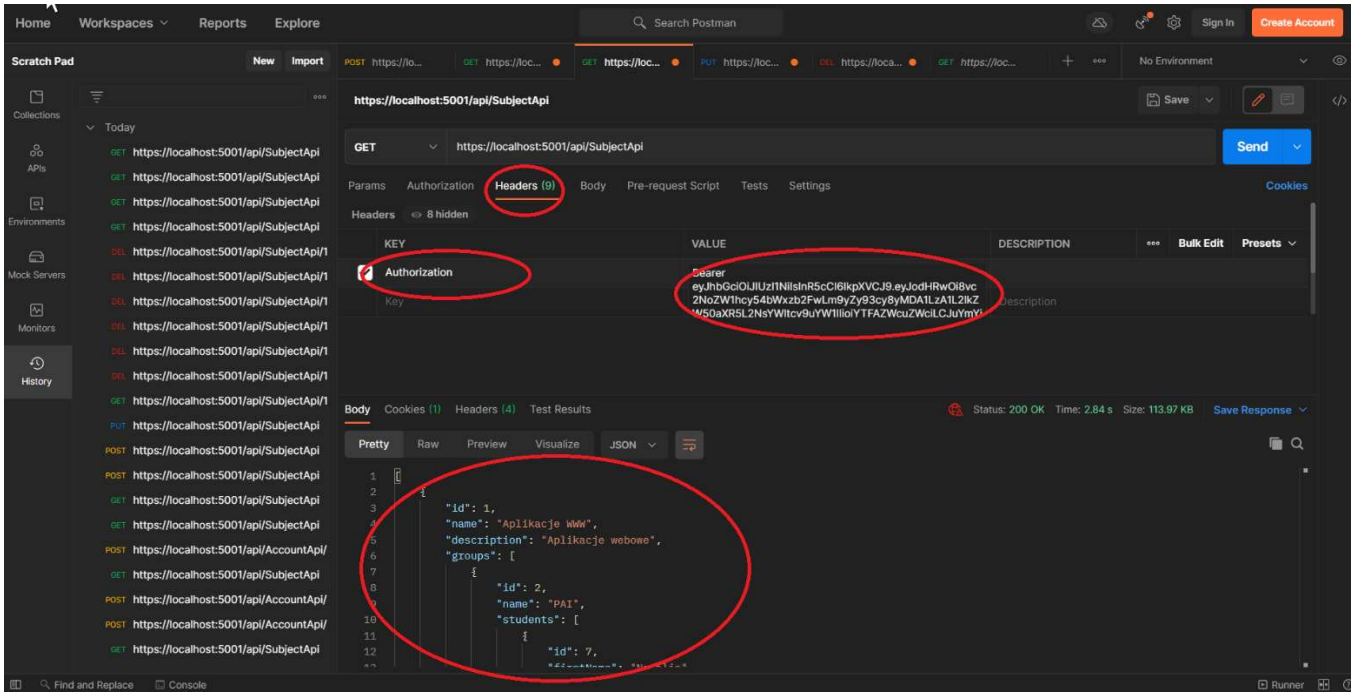
https://www.postman.com/downloads/

Po instalacji proszę przejsc do ustawień Postmana i wyłaczyc weryfikację SSL.



Od tego momentu możemy przygotowywać i wykonywać żądania HTTP poprzez Postmana. Jak można zauważyć nasze Web API wymaga authentykacji użytkowników podczas żądania. Inaczej niż w przypadku ASP.NET MVC, Web API jest bezstanowe (stateless, jedna z reguł REST), dzięki czemu do żądań wymagających authentykacji należy authoryzować się specjalnym tokenem (JwT Bearer Token), zawierającym tzw. Claims. Cały mechanizm jest zawarty w .NET Identity.





Możliwe jest również dodanie tokenu do Swaggera. Przykład poniżej.

AccountApi

POST /api/AccountApi/Login

Parameters | Cancel

No parameters

Request body | multipart/form-data

Login * required
string — a1@eg.eg

Password * required
string($password) — ••••••••

Execute | Clear

Responses

Curl

```
curl -X POST "https://localhost:5001/api/AccountApi/Login" -H  "accept: */*" -H  "Content-Type: multipart/form-data" -F "Login=a1@eg.eg" -F "Password=User1234"
```

Request URL

```
https://localhost:5001/api/AccountApi/Login
```

Server response

Code | Details

200

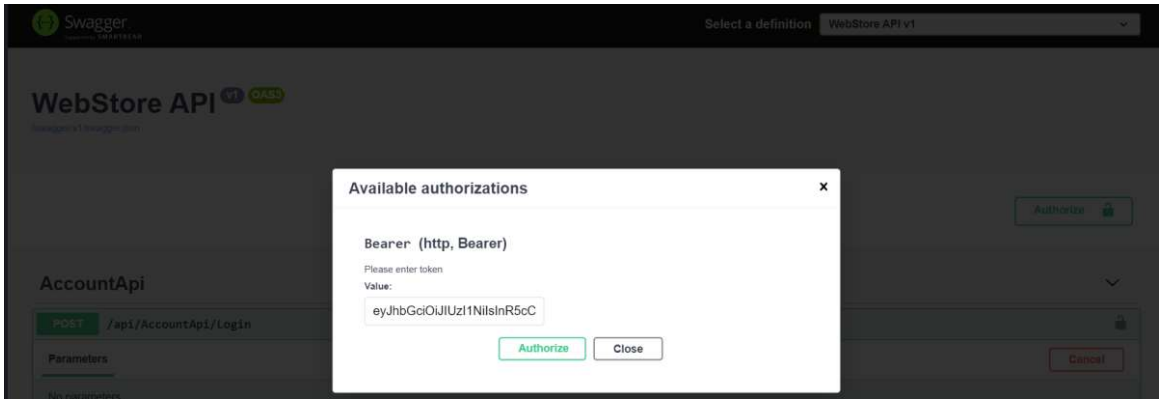
Response body


```
{
  "access_token":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJodHRwOi8vc2NoZW1hcy54bWxzb2FwLm9yZy93cy8yMDA1LzA1L2lkZW50aXR5L2NsYWltcy9uYW1lIjoiYTFAZWcuZWciLCJuYmYiOiIxNjM2MjQ4NTk2IiwiZXhwIjoiMTYzNjI1MzM5NiIsImh0dHA6Ly9zY2hlbWFzLm1pY3Jvc29mdC5jb20vd3MvMjAwOC8wNi9pZGVudGl0eS9jbGFpbXMvcm9sZSI6IkFkbWluIn0.o_V5d5j_84CfmCGcUb_iw4Eqf941Wg00ouMoJ6ge1lA",
  "expires_in": "2021-11-07T02:49:56"
}
```


Download

Response headers

```
cache-control: no-cache,no-store
content-length: 378
content-type: application/json; charset=utf-8
date: Sun07 Nov 2021 01:29:56 GMT
expires: Thu01 Jan 1970 00:00:00 GMT
pragma: no-cache
server: Kestrel
```



Address

GET /api/Address

Parameters | Cancel

No parameters

Execute | Clear

Responses

Curl

```
curl -X GET "https://localhost:44433/api/Address" -H  "accept: */*" -H  "Authorization: Bearer
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJodHRwOi8vc2NoZW1hcy54bWxzb2FwLm9yZy93cy8yMDA1LzA1L2lkZW50aXR5L2NsYWltcy9uYW1lIjoiZW1wMUBlZy5lZyIsIm5iZiI6IjE2NzM0NjIyMzciLCJleHAiOiIxNjczNDY3MDM3IiwiaHR0cDovL3NjaGVtYXMubWljcm9zb2Z0LmNvbS93cy8yMDA4LzA2L2lkZW50aXR5L2NsYWltcy9yb2xlIjoiU3RhdGlvbmFyeSBBcGkgVXNlciJ9.YyOAnQ189ikOf8NTKO1_Cys3P4X3WtpokYp5slHvgWM"
```

Request URL

```
https://localhost:44433/api/Address
```

Server response

Code | Details

200

Response body


```
[
  {
    "id": 1,
    "city": "Czestochowa",
    "zipCode": "42-200",
    "street": "Al. Armii Krajowej",
    "buildingNumber": 1,
    "apartmentNumber": 100,
    "country": "Poland"
  },
  {
    "id": 2,
    "city": "Czestochowa",
```

Zadanie

Po wykonaniu powyższej instrukcji proszę stworzyć pozostałe kontrolery API, aby uzyskać w pełni funkcjonalne zasoby Web API zgodne z funkcjonalnościami systemu.

- **OrderApiController**
- **InvoiceApiController**
- **StoreApiController**
- **AddressApiController**

Tak przygotowane API może stanowić warstwę **service backend** dla wielu aplikacji, np. mobilnych (IOS, Android, Flutter), desktopowych (WPF) czy webowych (SPA, React, Angular, Vue).

Przy wykorzystaniu utworzonych wcześniej usług (serwisów), należy pamiętać, aby NIE dodawać logiki biznesowej do kontrolerów - jest to zadanie oddelegowane do serwisów. Zadaniem kontrolerów jest:

- **Odebranie danych od klienta oraz ich walidacja,**
- **Przekazanie danych do serwisu – wywołanie metody z serwisu,**
- **Wyświetlenie danych otrzymanych z serwisu.**